

EECS 498 · AASE · FALL 2026

How Aider works

LAB01 · DESIGN YOUR PAIR-PROGRAMMER · SEP 14–15, 2026

Schedule

1. Aider deep dive
2. Hackathon 1
3. The next assignment
4. Office hours

The harness

Aider is a *harness*: a program that stands between you and a model.

The harness	The model
Picks the text the model sees	Reads it
Decides what a reply may do	Writes a reply
Edits files, then commits	Nothing

Take one apart now, build one later.

What the model never touches

- Your disk
- Your shell
- Your last session

Text in, text out. The harness does the rest.

What writes, and what does not

Arranges context

`/add`

`/read-only`

`/drop`

`/tokens`

`/ask`

`/diff`

Changes your files

`/code`

`/undo`

Bare text means `/code`. The default writes.

One request, three files

Add a `--priority` flag to taskr's add command.

taskr/cli.py

Parses the flag.

taskr/task.py

Holds the value.

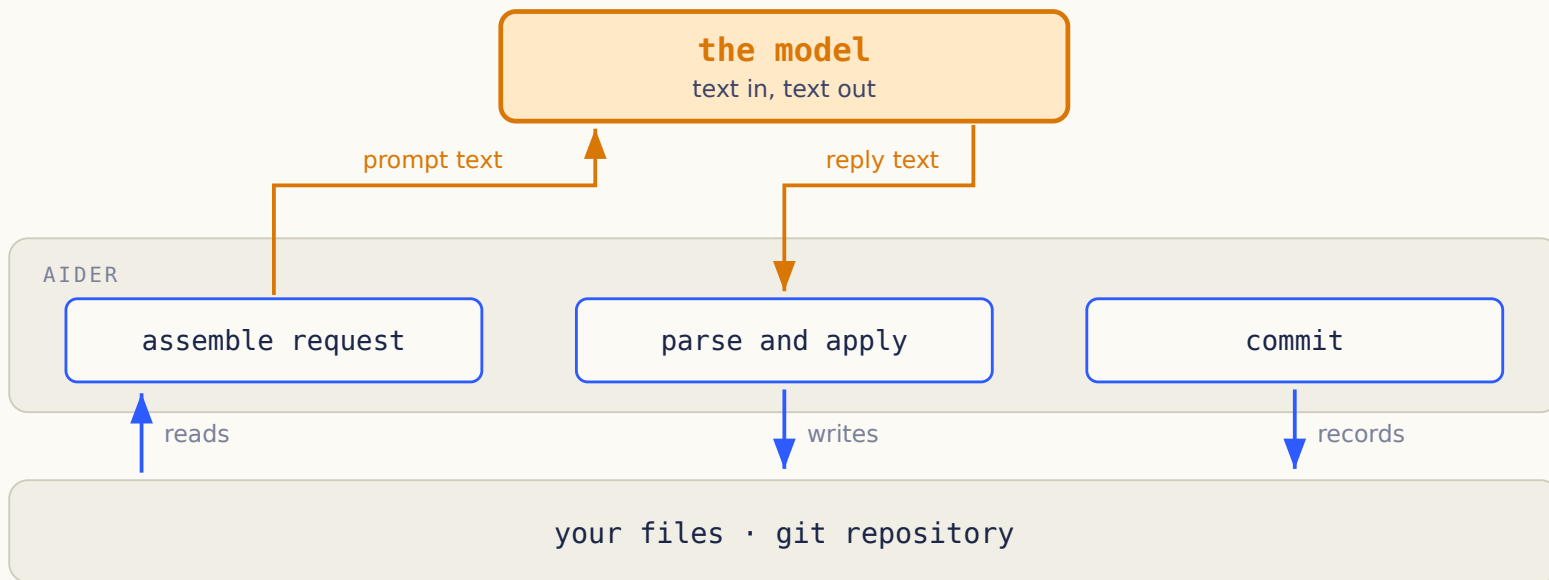
taskr/store.py

Saves and loads it.

One turn

SELECTED FILES TO COMMIT

Aider, the model, and the repo



The model touches no file.

What goes into the request

Fixed each turn	Instructions, added files, repo map
Grows each turn	Session history, your message

Context is a selection. A file you never added is not in it.

File selection with /add

```
/add taskr/cli.py taskr/task.py taskr/store.py  
/read-only specs/priority.md  
/tokens
```

Added files can be rewritten. Read-only files can only be cited.

Only these commands load a file.

The repo map

```
taskr/store.py  
    Store.add(title, tags) -> Task
```

```
taskr/task.py  
    Task: id, title, tags
```

Signatures, ranked into a budget. No function bodies.

One shared token budget



One budget: history, map, and files.

Edit-shaped replies

```
taskr/cli.py
<<<<<<< SEARCH
add.add_argument("title")
=====
add.add_argument("title")
add.add_argument("--priority", default="normal")
>>>>>>> REPLACE
```

Adds the flag. Stores nothing. Validates nothing.

The parse step

Parsing asks: is this a block?

A missing divider is a syntax error.

Validation asks: does it apply?

Unknown path, or zero exact matches.

Two questions. Two error messages.

Exact-match application

On disk

```
add.add_argument("title")
```

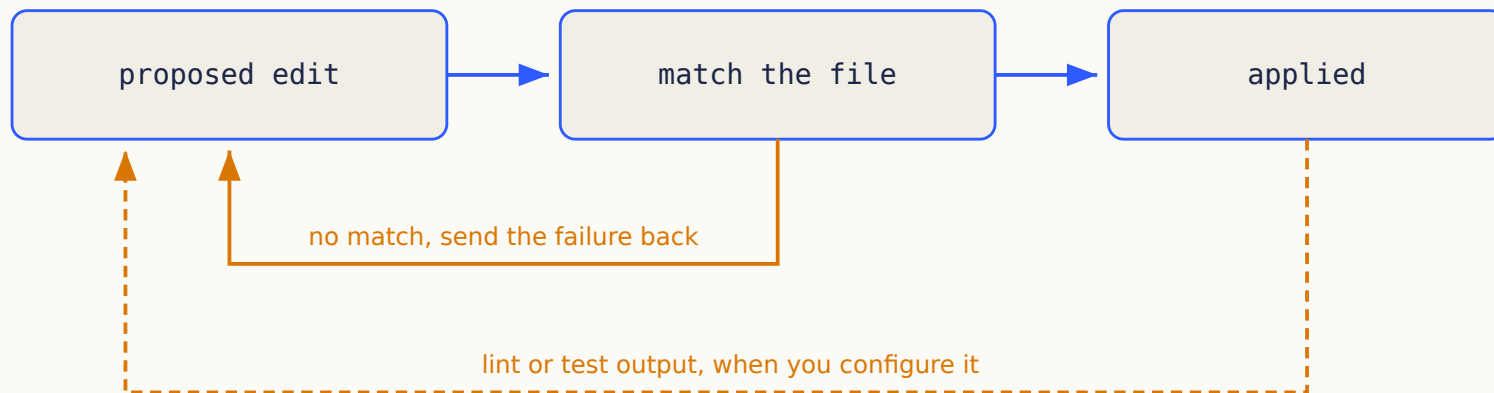
In the SEARCH block

```
add.add_argument( "title" )
```

Two extra spaces. Zero matches.

Your contract: exact matches only, all blocks or none.

The repair loop



A fixed cycle, not the model's choice.

One commit per edit set

```
* 8f2a1c3  aider: add --priority to the add command
* 41b9e07  aider: store priority on Task
* 0c7d5e1  initial taskr
```

Git reverses files. It does not reverse a sent email.

The six steps, recapped

With the person next to you:

1. What entered the context?
2. Which program read the reply?
3. What could look correct and be wrong?
4. What evidence proves it works?

Diagnosis

READ THE FAILURE BEFORE YOU RE-PROMPT

Three failure symptoms

Symptom	Check first
Calls a function that does not exist	The file was never added
Block looks right, will not apply	The file changed since
Tests pass, feature still broken	The tests missed the gap
The request times out	Endpoint, model, input size

One symptom, several causes. Find the evidence.

Better results, same model

A chat window

You paste in. You paste back. You hope.

Aider, same model

Reads what you selected, applies a fixed format, commits.

Fix the harness, then judge the model.

A feature that half works

```
$ taskr add "ship lab" --priority high
added #4
$ taskr list
#4  ship lab  priority: high

$ taskr list          # new process, same database
#4  ship lab  priority: normal
```

What do you inspect first?

THURSDAY SEPTEMBER 24

Hackathon 1

- Two hours, your own project
- Prompt revealed in the room
- Submit before you leave
- Graded separately from the build

Bring a runnable increment.

Build due **Friday September 25, 11:59 PM.**

The build

DESIGN FIRST, THEN IMPLEMENT IN INCREMENTS

What the packet contains

In the packet

```text SPEC ACCEPTANCE RUBRIC  
CHECKLIST GLOSSARY EXAMPLES DESIGN-  
GUIDE WORKFLOW + the Aider config ```

`CHECKLIST.md` lists what you hand in.

## Not in the packet

Application code. Tests. A layout. A diagram.

# The assistant's behavior

|                                  |                                            |
|----------------------------------|--------------------------------------------|
| <code>/files</code>              | → list selected files                      |
| <code>/add greet.py</code>       | → include this file in model requests      |
| <code>What does greet do?</code> | → stream an answer about the selected code |
| <code>Change Hello to Hi.</code> | → validate the reply and preview a diff    |

**A terminal assistant over selected files. Not all of Aider.**

# The approval step

| State               | <code>greet.py</code> | Git             |
|---------------------|-----------------------|-----------------|
| Diff shown, waiting | Hello                 | No commit       |
| Approved            | Hi                    | One commit      |
| Declined            | Hello                 | No commit       |
| Undone              | Hello                 | Reset to parent |

**One invalid block cancels the whole proposal.**

# The seven features

| Feature | Required behavior                                |
|---------|--------------------------------------------------|
| F1      | Conversation, budget handling, streamed replies  |
| F2      | Add, drop, list and clear context                |
| F3      | Current file contents, rendered consistently     |
| F4      | An edit-format prompt you have tested            |
| F5      | Parsing with useful errors                       |
| F6      | Exact edits, complete preview, explicit approval |
| F7      | One commit per edit set, guarded undo            |

# The assistant's config schema

```
base_url: https://api.example.edu/v1
model: course-model-id
api_key_env: ASSISTANT_API_KEY
context_budget: 8000
timeout_seconds: 60
temperature: 0.2
```

**Schema fixed. Loading and validation are yours.**

# Architecture ownership

## We specify

Observable behavior and safety rules

The API and configuration contract

Acceptance scenarios and the rubric

Where the course is going

## You design

Responsibilities and who owns state

Internal interfaces and error flow

Tests, fixtures, increment boundaries

Where later capabilities enter

Different architectures can earn full marks.

# Two sizes of spec

## **System spec, written once**

Behavior, architecture, interfaces, failures.

## **Increment spec, one per change**

Scoped context, ordered tasks, a checkable result.

Commit the design before the code.

# The three diagrams

| Diagram                  | Answers                     |
|--------------------------|-----------------------------|
| Components and data flow | Who owns state?             |
| Request sequence         | Who does what in one edit?  |
| Edit lifecycle           | When can an edit be undone? |

Mermaid is enough. Commit the first versions.



# Rubric: the three buckets

| Group                       | Points |
|-----------------------------|--------|
| Specification and diagrams  | 40     |
| Behavior and verification   | 40     |
| Evidence and reconciliation | 20     |

`RUBRIC.md` carries the split, and grades you.

# Model rules for specifications

- Drafting, critique, diagrams, code, tests
- 9B recommended, 4B permitted
- Secondary models recorded too
- Any compatible host allowed

**A different provider is not a different model.**

THROUGH DECEMBER

# One repository to December



**One small checkable result first. Keep the phase snapshots.**

# What comes next

THE SAME REQUEST, IN A DIFFERENT KIND OF TOOL

# Claude Code on the same request

| Aider                    | An agentic CLI                |
|--------------------------|-------------------------------|
| You frame a bounded task | You hand over a broad one     |
| You select the files     | It discovers them             |
| A fixed repair cycle     | The model picks the next tool |
| You check between tasks  | Permissions bound it          |

# Harness versus instructions

- Project instructions: persistent guidance
- Skills: instructions for one task
- Hooks: checks attached to events
- Integrations: actions and results

**Prose asks. Code and permissions enforce.**

# Boundaries for later capability

- Where could a model-chosen tool enter?
- Could you swap the endpoint client?
- Could a web interface reuse this?

Answer with a boundary, not an implementation.

# Office hours

THE REST OF THE SESSION IS YOURS



# Questions?